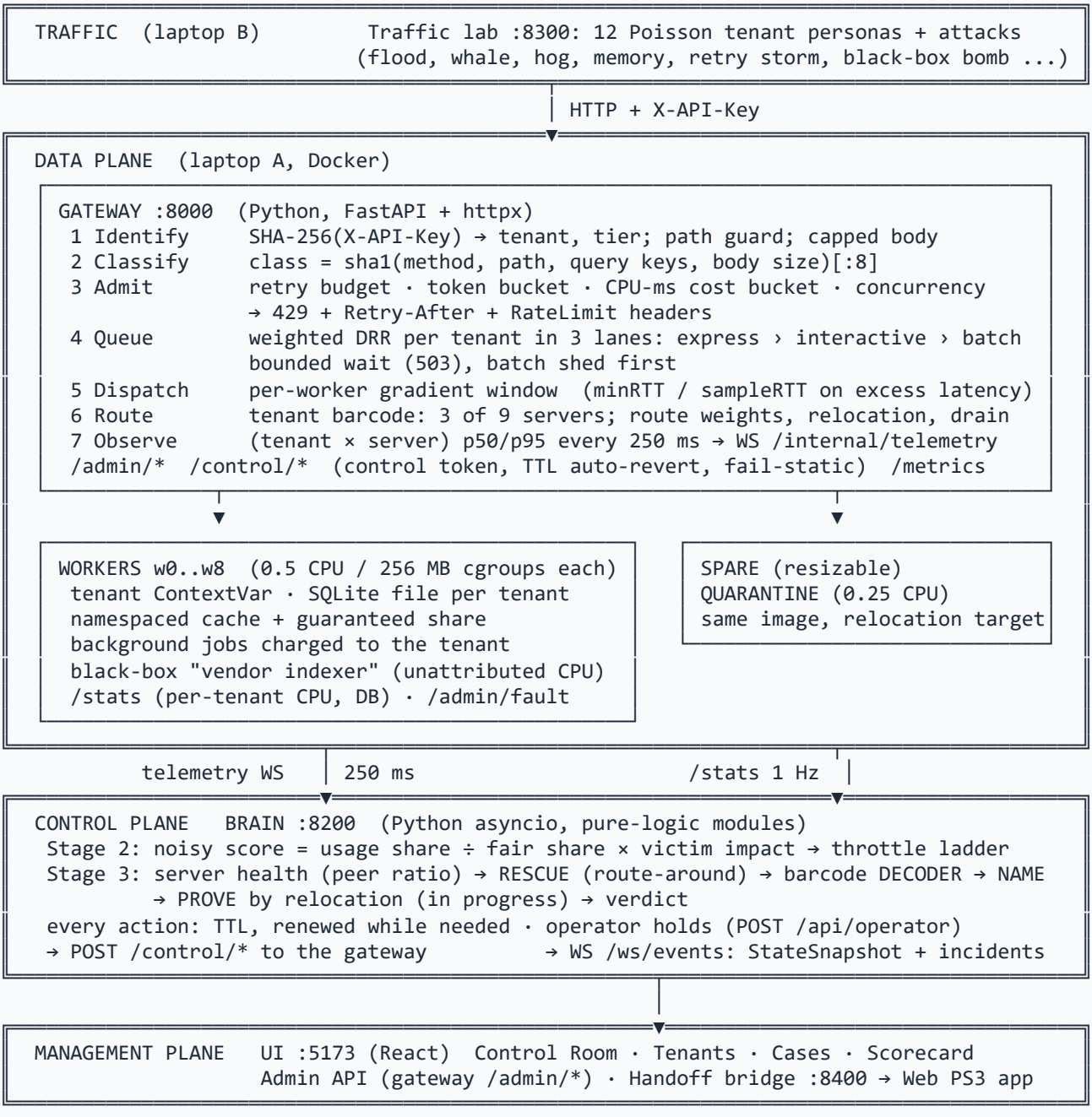
TenantGuard

Multi-Tenant Isolation & Noisy-Neighbor Defense · CodeAstra 2.0 · Cloud Computing & Distributed Systems · PS 3

Architecture | Tech Stack | Project Flow

> *"TenantGuard gives every tenant of a shared service its fair share, and names the tenant behind any slowdown, even when the expensive work is invisible to a rate limiter."*

Version 1: Technical Architecture



The 3 Planes

Plane	Components	Job
Data	gateway, w0..w8, spare, quarantine	Serve every tenant request: identify, admit, queue fairly, route by barcode, measure
Control	brain	Watch telemetry + worker stats; rescue victims, name the culprit, throttle with TTLs, explain every action
Management	admin API, UI, handoff bridge, lab, demo driver	Operators see and steer it: modes, quotas, cases, attacks, scorecard

The 5 Layers of Isolation (not just a rate limit)

Layer	Name	What it does	Where
L1	Request	Identity, token bucket, retry budget: a flood or retry storm gets 429 with an accurate Retry-After	`gateway/admission.py`, `retry.py`
L2	Cost	A CPU-ms bucket debited with the CPU each request really used (measured by the worker)	`gateway/buckets.py`
L3	Scheduler	Weighted DRR in 3 lanes behind an adaptive per-worker window: cheap requests never wait behind a whale	`gateway/scheduler.py`, `limiter.py`
L4	Kernel	cgroup CPU / memory / pids limits per worker container; spare and quarantine pools	`docker-compose.yml`
L5	Data	One SQLite file per tenant; namespaced cache with a guaranteed share; per-tenant memory budget	`worker/app/store.py`, `cache.py`
+ Adaptive loop		The brain: rescue, name, throttle, recover. It works even when the harm is invisible to L1 and L2.	

Components & Tech Stack

Component	Port	Built with	Notes
Gateway	8000	Python 3.12, FastAPI, httpx (pooled), uvicorn	All state in memory (one gateway); Prometheus `/metrics`
Workers ×9 + 2	8100	Python, FastAPI (sync handlers), SQLite, psutil	`time.thread_time()` per request = exact handler CPU
Brain	8200	Python asyncio, websockets, httpx, Pydantic	Pure-logic modules (score, ladder, health, rescue, decoder) tested without I/O
Traffic lab	8300	Python asyncio + httpx keep-alive pools	Writes `lab/results/*.json`: the only source of scorecard numbers
UI	5173	React 18, Vite, TypeScript, Tailwind, Recharts	WS from the brain at 2 Hz; mock mode for offline development
Handoff bridge	8400	Python, FastAPI, websockets	Web PS3 convergence: incidents ↔ cases
Contracts + config		Pydantic models in `common/`, `tenantguard.yaml`	One shared schema; every tunable in YAML
Infra		Docker Compose, cgroup v2, two laptops over Tailscale	No external network at runtime

Adaptive Throttling Ladder (Stage 2)

Level	Enters when	Action (always with a TTL)
normal	-	Full plan
warn	noisy score ≥ 1.5 and neighbours hurt, held 2 s	Incident opened, nothing throttled
throttle	score ≥ 2.0, held 2 s	Quota × 0.5 per step (AIMD), via LimitsCmd
deprioritize	still noisy, held 4 s	Scheduler weight × 0.25
quarantine	score ≥ 4.0, held 6 s	Quota × 0.1 and weight × 0.1; a block needs human approval (handoff case)
recover	score < 1.0, held 10 s	+0.1 factor per step back to the full plan

Thresholds and hold times come from `tenantguard.yaml` (`brain.noisy`). Rescue comes first: the ladder acts only if harm persists. A legit burst that hurts nobody is never throttled.

Stage 3: Rescue, Name, Prove

Step	How	Status
Health	Each server's p95 compared with its peers (median over its tenants, so a whale's own slow requests don't count)	done
Rescue	Route weights move victims off slow servers onto their healthy ones, keeping every server under 70% CPU	done
Name	AG(2,3) barcodes: any two tenants share at most one server, so the set of slow servers decodes to the culprit (with a confidence) or to a host fault with no tenant blamed	done
Prove	Move the suspect to the spare: the original servers recover and the slowdown follows it (R = 1 − post/pre slowdown)	in progress
Verdict	costly pattern · over plan · honest growth · host fault · mitigated / no blame	in progress

Safety

Rule	How
Auto-undo	Every control override has a finite TTL (≤ `gateway.ttl_max_s`) and reverts by itself; the brain renews it while needed
Fail-static	If brain heartbeats stop for 2 s, the gateway keeps its last config while TTLs still expire. A dead brain never leaves a tenant punished
Humans and ladder agree	Operator release / extend go to the brain (`OperatorCmd`), so the ladder obeys them instead of fighting them
Hardened edges	Control token on `/admin/*`, `/control/*`; path guard (no traversal to worker admin); bounded TTLs and quotas; capped request bodies

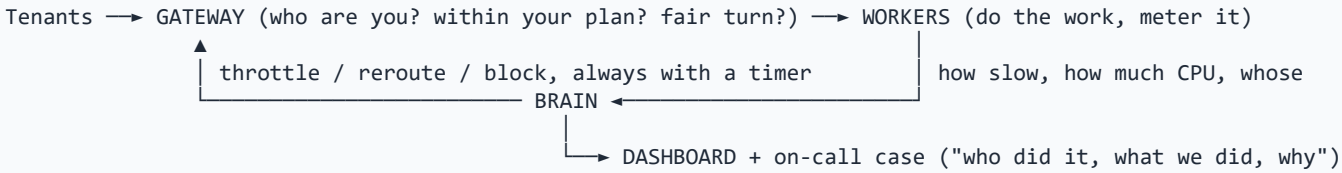
Measured: why a rate limiter is not enough

Endpoint	Handler CPU (what request metering sees)	Background CPU	Black-box indexer CPU
`GET /api/light`	0.396 ms	0	0
`POST /api/write`	0.185 ms	1.429 ms	0
`POST /api/ingest`	**0.066 ms**	0	**20.25 ms**

Source: `worker/calibration.json`. Docker host, one worker limited to 0.5 CPU, sequential client, mean per request. The OFF / Industry / TenantGuard scorecard (victim p99 per attack) is to be measured; it will come only from `lab/results/*.json`.

Version 2: Simple Project Flow (Team View)

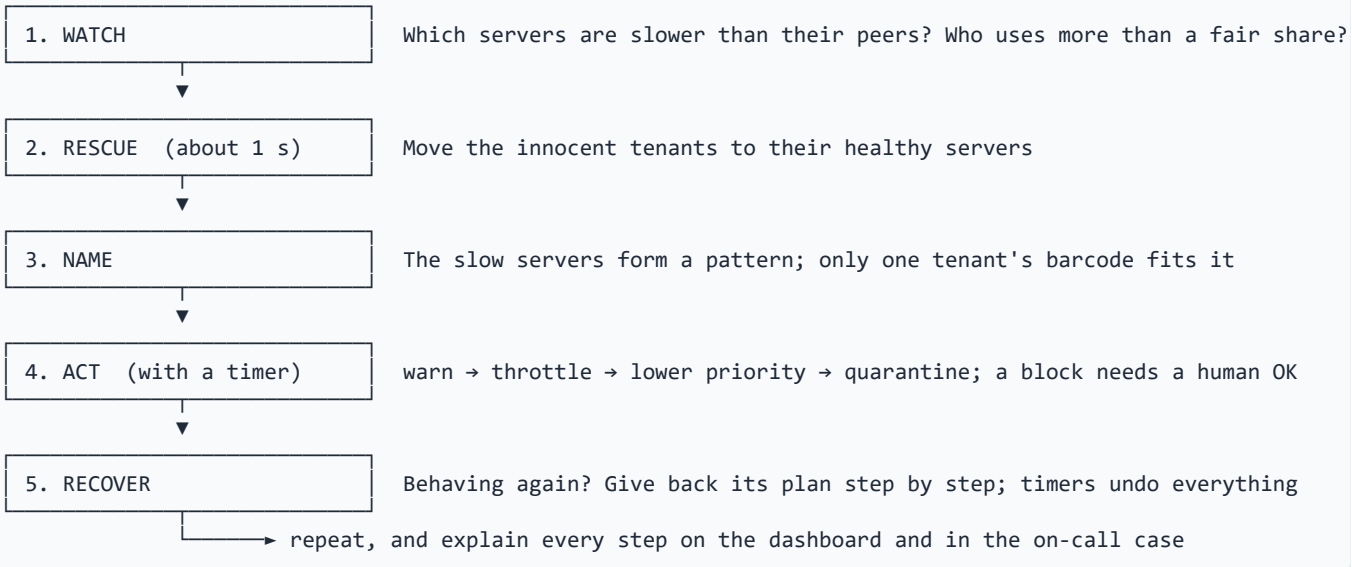
The Big Picture in One Line



A Request's Journey, Step by Step

1. Tenant K sends POST /api/ingest with its API key
2. Gateway: "You are K (Pro plan)." Unknown key? → 401
3. Within your plan? rate ✓ CPU budget ✓ in-flight ✓ not a retry storm ✓ (else 429 + "retry in 2 s")
4. Wait your fair turn: cheap requests take the express lane; nobody waits more than 1 s (else 503)
5. Go to one of K's 3 servers (its "barcode"), avoiding slow or drained ones
6. Worker does the work in K's own database and charges the CPU to K
   ...but the black-box indexer it triggers burns 300× more CPU that nobody bills
7. Gateway records the latency for (K, server) and reports it to the brain every 250 ms

The Automatic Protection Loop (4 times a second)



Which Box Does What

Box	In one sentence
Gateway	The front door: knows every tenant, enforces its plan, keeps the queue fair, and obeys the brain's timed orders.
Workers	The shared app: does the work, keeps each tenant's data apart, and measures exactly what each tenant costs.
Brain	The detective: spots slow servers, rescues victims, names the culprit, and turns the throttle up or down.
Dashboard	The control room: every tenant, every server, every incident and the reason for every action, live.
Lab	The crowd: 12 realistic tenants plus attacks, and the measured scorecard.
Handoff bridge	Turns each incident into a case for the on-call engineer and carries their decision back.

The Demo in Pictures (4 minutes, `tools/demo.py`)

Act	What the judges see
Calm	12 green tiles, fairness near 1
OFF + flood	Everyone sharing K's servers turns red: the noisy-neighbour problem
Industry	The flood is refused (429). Then the black-box bomb: the rate limiter sees nothing and the victims turn red
TenantGuard	Same bomb: victims rescued, K named with a confidence, the case appears for the on-call engineer
Faulty server	One server goes bad by itself: host fault, no tenant blamed
Honest spike	An Enterprise tenant's sale day: a genuine burst, not throttled
Scorecard	OFF vs Industry vs TenantGuard victim p99 (to be measured; from `lab/results`)

Presentation tip: Put Version 1 on the architecture slide, then walk the judges through it using the Version 2 story ("let's follow one request from K..."). Judges remember stories better than boxes.